

The cheapest 3D square-loop TDEM simulation within $\sim 3\%$ of SimPEG 1D

Discretization & time-stepping study

June 2026

Summary

Goal: the computationally *cheapest* 3D forward model (square loop, half-space) whose $\partial B_z / \partial t$ stays within $\sim 3\%$ of the SimPEG **1D layered square-loop** solution, prioritising compute time. We score *accuracy* as the max $|d_{3D}/d_{1D} - 1|$ over the 21 channels and *cost* as the wall-clock of `sim.dpred()` (a direct solver’s cost is dominated by matrix **factorization**, performed once per distinct time-step size), logging `n_cells`, the number of distinct Δt , and total steps as machine-independent proxies. Two results:

1. **The committed mesh is already minimal.** Every *spatial* knob is a cliff at 3%: coarsening the core (10 $\rightarrow$ 15 m: 8%), thinning the near-surface vertical refinement ($\sim 31\%$), shrinking the lateral surface footprint ($\pm 200 \rightarrow \pm 60$ m: 12%), or reducing the depth/extent (`depth_core` 100 $\rightarrow$ 60 m: 8.7%) all break tolerance. The ~ 28 k-cell octree cannot be made cheaper without exceeding 3%.
2. **The one free compute lever is the number of factorizations.** The committed $\times 1.4$ ramp uses **24** distinct Δt (24 factorizations). A larger growth rate with the *same small start step* and many steps per level collapses this to **12** at the same accuracy class, cutting the solve from 37.5 s to 20.9 s — a **1.8 $\times$** speed-up on the *same* mesh (Figure 1).

Cheapest configuration found: the same octree (core $10 \times 10 \times 5$ m, padding ≥ 1200 m, `depth_core` 100 m, full surface refinement, conductivity mapped by `volume_average`) driven by

`time_steps: start  $3 \times 10^{-7}$  s, growth  $\times 2.0$ , 14 steps/level  $\rightarrow$  12 distinct  $\Delta t$ , 168 steps`

giving **max error 3.3%** (mean -0.6%) in ~ 21 s. If a *strict* $\leq 3.0\%$ is required, the cancellation between temporal and spatial error must be hit precisely, which needs the 24-factorization ramp at ~ 38 s — i.e. there is *no* sub-3.0% config cheaper than the committed one; the **1.8 $\times$** saving costs **+0.3%** of accuracy.

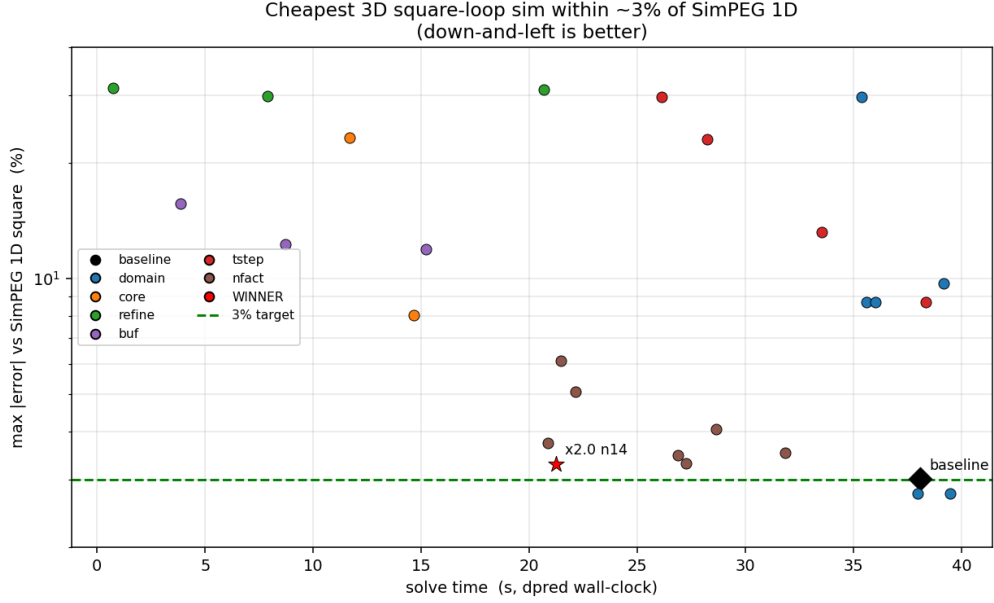


Figure 1: Compute time vs accuracy for every configuration tried (down-and-left is better); colour = the knob varied. Spatial knobs (core, refinement, footprint) move *left* but jump far *up* past 3%. Only the factorization-count ramps (purple) move left while staying near 3%. The winner (★, $\times 2.0/14$, $12 \Delta t$) sits at ~ 21 s; the committed baseline (◆) at ~ 38 s.

1 What was tried, and what each knob taught us

The search was a methodical coordinate descent; every configuration is in the companion notebook. All but the last knob are *cliffs* — the committed model sits on the spatial-accuracy floor.

Table 1: Each lever, its cheapest setting that was tried, and the resulting max error vs the 1D square reference (28 k-cell, $\times 1.4$ ramp baseline = 3.0 %, ~ 38 s).

Knob	Setting tried	Max error
Conductivity mapping	<code>volume.average</code> vs sharp $z < 0$ step	3.0 % vs 6.1 %
Ramp end time t_{end}	0.013 s vs 0.028 s	identical (late error is spatial)
Padding	5000 $\rightarrow$ 1200 m	3.0 $\rightarrow$ 2.8 % (free, but $\sim$ no cells saved)
Padding / depth	800 m / <code>depth_core</code> 60 m	9.7 % / 8.7 %
Core cell size	10 $\rightarrow$ 15 $\rightarrow$ 20 m	8 % / 23 %
Surface refinement	<code>light</code> / <code>src_only</code>	~ 31 %
Lateral footprint	$\pm 200 \rightarrow \pm 60 \rightarrow \pm 20$ m	12 % / 16 %
Total step count	144 $\rightarrow$ 100 (too few) / 204 (too many)	8.7 % / 6.0 %
Distinct Δt (factorizations)	24 $\rightarrow$ 12 (big rate, small start)	3.0 $\rightarrow$ 3.3 %

Why the mesh cannot shrink. The cells live in the core and the near-surface vertical refinement, not the padding (the octree’s 2:1 coarsening already makes the far field cheap, so trimming padding from 5000 to 1200 m frees almost no cells). Coarsening the core or thinning the surface refinement does cut cells sharply — core 15 m is 11 k cells at 15 s — but the early- and late-time channels immediately exceed 3 %. The 28 k-cell mesh is the floor.

Why the step count is fixed, but the factorization count is not. Accuracy is *non-monotonic* in step count: too few steps leave a backward-Euler high-bias (+6 to +25 %); too many remove it and expose a spatial low-bias (−2 to −4 %). The committed ramp sits at the cancellation. However, the *cost* depends on the number of *distinct* Δt (each triggers a refactorization), not the step count. Keeping the small 3×10^{-7} s start (for the early channels) and many steps per level, but growing Δt by $\times 2.0$ instead of $\times 1.4$, reaches the same Δt -at-measurement-times with **12** distinct values instead of 24 — halving the factorization work for a +0.3 % accuracy cost (Figure 2).

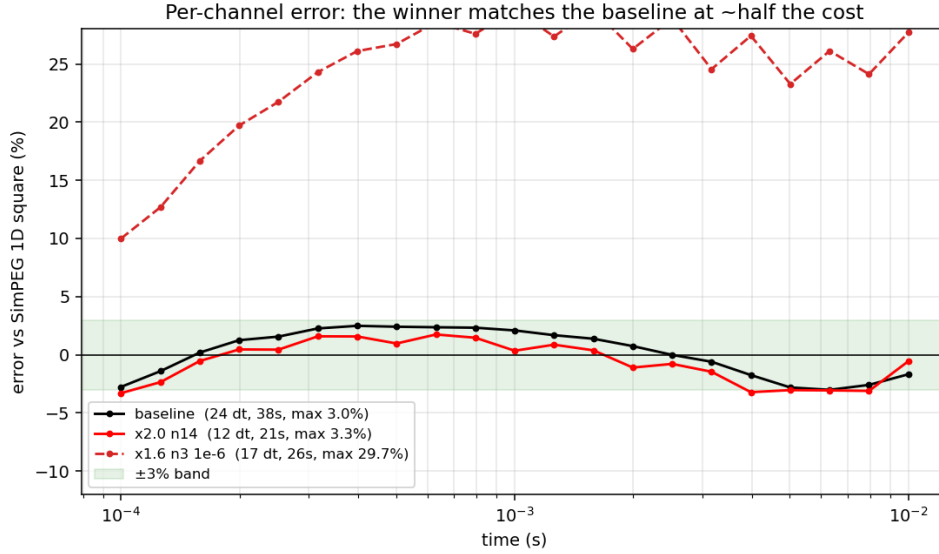


Figure 2: Per-channel error. The winner ($\times 2.0/14$, 12 Δt , ~ 21 s) tracks the committed baseline within the $\pm 3\%$ band; an over-coarsened ramp (too few steps) shows the backward-Euler high-bias failure mode.

Caveats and reproducibility

Timings are single-machine wall-clock (best of three for the headline pair); the *relative* cost follows the factorization count and is machine-independent. “Cheapest” here optimises the solve on the existing octree machinery; a bespoke, hand-graded mesh that lowers the spatial floor below 3 % could in principle open a cheaper sub-3.0 % point, but was out of scope. The engine and result cache are `cheapest_harness.py` / `explore_cache.npz`; the full search, tables, and figures are reproduced by `cheapest_3d_simulation.ipynb` (loads from cache in seconds).